

FORMATION EMBARQUÉE

TD 10 Stm32

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 10 — STM32 : énoncés et corrigés détaillés

Cible : Nucleo-F411RE (adapter les broches sinon). Les corrigés montrent le code utilisateur à placer entre les balises `USER CODE` d'un projet CubeMX ; la configuration CubeMX est décrite à chaque fois.

Exercice 1 — Chenillard cadencé par TIM3, vitesse par ADC

Énoncé. 4 LED avancent au rythme d'une interruption TIM3 ; le potentiomètre (ADC) règle la vitesse. Zéro `HAL_Delay` dans la boucle.

Corrigé

Config CubeMX : PA5-PA8 en GPIO_Output ; PA0 en ADC1_IN0 (résolution 12 bits, mode continu OFF) ; TIM3 : horloge interne, PSC = 9999, ARR = 999 (à 100 MHz APB1 timer clock → $100 \text{ MHz} / 10000 / 1000 = 10 \text{ Hz}$ de base), interruption « update » activée dans NVIC.

```

/* USER CODE BEGIN PV */
static const uint16_t LED_PINS[4] = {GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7, GPIO_PIN_8};
static volatile uint8_t index_led = 0;      // écrit en ISR, lu partout
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
// ISR de débordement TIM3 : avancer le chenillard. COURTE : 3 écritures GPIO.
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) {
        HAL_GPIO_WritePin(GPIOA, LED_PINS[index_led], GPIO_PIN_RESET);
        index_led = (index_led + 1u) & 0x03u;    // modulo 4 par masque
        HAL_GPIO_WritePin(GPIOA, LED_PINS[index_led], GPIO_PIN_SET);
    }
}

static uint16_t lire_pot(void)
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);
    uint16_t v = (uint16_t)HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return v;                                // 0..4095
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim3);              // démarrer le timer + IRQ
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    // La vitesse se règle en modifiant ARR : période = (PSC+1)(ARR+1)/f_tim.
    // pot 0..4095 → ARR 199..4095+199 → ~2 Hz à ~25 Hz d'avancement.
    uint16_t pot = lire_pot();
    __HAL_TIM_SET_AUTORELOAD(&htim3, 199u + pot);
    HAL_Delay(50);    // toléré ICI : ne cadence rien, il espace les lectures ADC
}
/* USER CODE END WHILE */

```

Points de correction : le rythme vient du **matériel** (TIM3), pas d'une attente logicielle — coupe le debugger en pause : le chenillard continue ; **volatile** sur **index_led** ; modification d' **ARR** à chaud (le registre « auto-reload » est fait pour ça).

Exercice 2 — Console UART : réception IT + ring buffer + commandes

Énoncé. Réception par interruption dans un tampon circulaire ; commandes **pwm <0-100>** et **status** , avec écho.

Corrigé (extraits essentiels — réutilise le ring buffer du TD 01)

Config CubeMX : USART2 115200 8N1, interruption activée ; TIM2 CH1 (PA5) en PWM, PSC/ARR pour 1 kHz avec ARR = 999 (rapport en ‰ $\approx \% \times 10$).

```

/* USER CODE BEGIN PV */
static RingBuffer rb_rx;           // le ring buffer du TD 01, tel quel
static uint8_t octet_rx;          // tampon d'1 octet pour la HAL
static char ligne[32];
static uint8_t ligne_pos = 0;
static uint8_t pwm_pct = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
// ISR : UN octet reçu → on le pousse dans le ring buffer et on réarme.
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2) {
        (void)rb_put(&rb_rx, octet_rx);           // plein ? octet perdu, tant pis
        HAL_UART_Receive_IT(&huart2, &octet_rx, 1); // RÉARMER : sinon plus rien
    }
}

static void traiter_ligne(const char *l)
{
    int v;
    if (sscanf(l, "pwm %d", &v) == 1 && v >= 0 && v <= 100) {
        pwm_pct = (uint8_t)v;
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, v * 100); // ARR=999 → %
        printf("OK pwm=%d%\r\n", v);
    } else if (strcmp(l, "status") == 0) {
        printf("pwm=%u%% uptime=%lums\r\n", pwm_pct, (unsigned long)HAL_GetTick());
    } else if (l[0] != '\0') {
        printf("ERR commande inconnue\r\n");
    }
}

// Redirection de printf vers l'UART (newlib de ST)
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 10);
    return ch;
}

/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
rb_init(&rb_rx);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
HAL_UART_Receive_IT(&huart2, &octet_rx, 1); // amorcer la réception
printf("console prete\r\n");
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    uint8_t c;
    while (rb_get(&rb_rx, &c)) { // vider ce qui est arrivé
        HAL_UART_Transmit(&huart2, &c, 1, 10); // écho
        if (c == '\r' || c == '\n') {
            ligne[ligne_pos] = '\0';

```

```

        traiter_ligne(ligne);
        ligne_pos = 0;
    } else if (ligne_pos < sizeof ligne - 1) {    // TOUJOURS border
        ligne[ligne_pos++] = (char)c;
    }
}
// ...la boucle reste disponible pour le reste de l'application...
}
/* USER CODE END WHILE */

```

Points de correction : le réarmement dans le callback (l'oubli = « je ne reçois que le premier caractère », bug archi-classique) ; **l'ISR ne parse pas** — elle stocke, la boucle parse (règle « ISR courte ») ; la ligne bornée ; la validation `0..100` avant d'appliquer.

Exercice 3 — Mesure de fréquence par Input Capture

Énoncé. Mesurer la fréquence d'un signal carré (reboucler le PWM de l'exercice 2 sur l'entrée !), affichage UART.

Corrigé

Config CubeMX : TIM4 CH1 (PB6) en « Input Capture direct mode », front montant, PSC = 99 (→ compteur à 1 MHz : 1 tick = 1 µs), ARR = 65535, interruption capture activée. Relier PA5 (PWM) à PB6 par un fil.

```

/* USER CODE BEGIN PV */
static volatile uint32_t periode_us = 0;
static volatile uint8_t mesure_prete = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    static uint16_t capture_prec = 0;
    if (htim->Instance == TIM4 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
        uint16_t capture = (uint16_t)HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
        // Soustraction sur uint16_t : correcte MÊME si le compteur a débordé
        // entre les deux fronts (arithmétique modulo 65536 – cf. millis() !)
        periode_us = (uint16_t)(capture - capture_prec);
        capture_prec = capture;
        mesure_prete = 1;
    }
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_TIM_IC_Start_IT(&htim4, TIM_CHANNEL_1);
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    if (mesure_prete) {
        mesure_prete = 0;
        uint32_t p = periode_us; // copie locale
        if (p > 0)
            printf("f = %lu Hz\r\n", (unsigned long)(1000000UL / p));
    }
    HAL_Delay(200);
}
/* USER CODE END WHILE */

```

Points de correction : le prescaler choisi pour un tick « rond » (1 μ s : le calcul de fréquence devient trivial et la plage mesurable — période max 65,535 ms $\rightarrow$ ~15 Hz mini — doit être **énoncée**) ; la soustraction non signée qui absorbe le débordement ; la mesure faite par le **matériel** au front près (aucun jitter logiciel, contrairement à un comptage dans une ISR GPIO).

Exercice 4 — Provoquer et analyser un HardFault

Énoncé. Déréférence un pointeur invalide, observe, explique.

Corrigé

```

volatile uint32_t *p = (uint32_t *)0xDEADBEEF; // adresse non mappée
*p = 42; // 💥 BusFault  $\rightarrow$  HardFault

```

Ce qu'on observe : le débogueur s'arrête dans `HardFault_Handler()` (boucle infinie générée par CubeMX). Le **Fault Analyzer** de CubeIDE (fenêtre *Fault Analyzer* en session de debug) décode les registres système :

- `CFSR` : le bit **PRECISERR** (BusFault précis) est levé ;
- `BFAR` contient **0xDEADBEEF** — l'adresse fautive exacte ;
- la pile empilée par le Cortex-M donne le **PC fautif** → double-clic ramène à la ligne `*p = 42;` .

Explication attendue : à l'exception, le Cortex-M empile automatiquement R0-R3, R12, LR, PC, xPSR ; les registres de faute (CFSR/HFSR/BFAR/MMFAR) disent *quoi* et *où*. C'est toute la différence avec l'AVR, qui redémarre sans laisser d'indice. En production, on écrit un handler qui sauvegarde ces registres (en RAM non initialisée ou en flash) puis redémarre — le crash devient diagnosticable après coup.

Variantes à essayer : lecture à une adresse non alignée castée en `uint32_t*` (selon la config, UsageFault UNALIGNED), appel d'un pointeur de fonction nul, débordement de pile (récursion) — chacun signe différemment dans le CFSR.

Auto-évaluation avant le TP 5

Sans notes : calculer PSC/ARR pour une fréquence donnée ; expliquer le réarmement de `HAL_UART_Receive_IT` ; pourquoi l'Input Capture bat toute mesure logicielle ; citer 3 registres de diagnostic d'un HardFault et leur rôle.

➡ Passe au **TP 5 — Station météo STM32 + FreeRTOS**.